Parallel Programming · Winter term 2026/27

Exercise 1



Recommended Workflow

In the exercises, we work on the *current* sheet.

We recommend creating a Git repository to store and maintain all files you create throughout the exercises. Fulda University of Applied Sciences provides a GitLab instance that is available for free use, see `https://www.hs-fulda.de/hlsb/en/research-publishing/research-data-management-1/gitlab`. **Do not store your files locally on the lab PCs, they may be deleted after reboot!**

All files in your Git repository remain private and are accessible only to you (unless you choose to share them). You can access your repository from home and continue working.

For each exercise, create a dedicated folder containing a `README.md` and a `Makefile`. This structure will help you to revisit and reproduce your work later without issues. Note that the exercises build upon each other, so you should keep your solutions from previous weeks available and up to date.

When programming locally on the lab PCs, we recommend using the terminal, especially for compilation. This is important because later in the semester we will work on an HPC cluster, where the terminal will be our primary interface. Locally, however, you may additionally use IDEs such as CLion or VSCode if you prefer.

Task 1 · `pi.c`

Write a *sequential* `C` program, named `pi.c`, that calculates an approximation of Pi (π) using the *Leibniz formula*[1] in a loop:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \dots \tag{1}$$

The number of iterations should be provided as a command-line argument.

Task 2 · `pi.c`

Try different numbers of iterations for `pi.c` and observe how many decimal places of π are computed correctly.

a) Measure the program's execution time using the `time` command.
b) Explain what the different time values displayed in the terminal represent.

Task 3 · `pi_threads.c`

Write a new program `pi_threads.c`. As before, it approximates π, but this time using POSIX threads (*pthreads*)[2]. Describe your program and analyze the advantages and disadvantages of using pthreads. Test how the program's execution time changes with different numbers of iterations (sequential baseline vs. various numbers of POSIX threads).

Tip: There are multiple algorithmic approaches to parallelizing the Leibniz sum, which may lead to different performance characteristics.

[1] `https://en.wikipedia.org/wiki/Leibniz_formula_for_%CF%80`
[2] `https://www.man7.org/linux/man-pages/man0/pthread.h.0p.html`

Task 4 · AI Chat with HAWKI (optional)

Attention

AI tools are recommended, but the work must be your own. We recommend using AI tools such as HAWKI or AI coding agents for the exercises, e.g., to have your code explained and reviewed. They are **not necessary**: you can complete all tasks and pass the exam without them. You remain fully responsible for everything you hand in or present: every line of code, every measurement, and every statement, including those an AI tool suggested. AI output can be wrong even when it sounds convincing, so check it and use only what you understand. **AI tools are not available in the oral exam**, so you must be able to explain code and results on your own.

These rules follow the principles of good research practice. The German Research Foundation (DFG) states that generative AI may be used, but that its use does not relieve anyone of their responsibility for the content, that it must be disclosed (which tool, for what purpose, to what extent), and that only humans can be authors: `https://www.dfg.de/de/grundlagen-themen/digitale-themen/ki`.

HAWKI: the AI Chat of HS Fulda

HAWKI is a chat interface for large language models, similar to ChatGPT. It was developed by the HAWK university of applied sciences (Hildesheim, Holzminden, Göttingen), and HS Fulda provides it free of charge to all its members: `https://ki-chat.hs-fulda.de`. Log in with your fd ID. HAWKI is only reachable from the university network; from home, connect to the HS Fulda VPN first. Your prompts are passed to the language model without your name, so no personal profile is created. Still, **never enter personal data, passwords, or other confidential information.** More information: `https://www.hs-fulda.de/en/our-university/a-z-all-institutions/office-for-teaching-and-learning/ai-supported-text-creation-1`.

Use HAWKI as a *tutor* for your own code, not as a code generator. A chat only sees what you paste into it: it cannot read your files or compile and run your code. (In Exercise 2, you will try an AI coding agent that works directly on your files.) This task is optional.

Tasks

a) Paste your `pi_threads.c` into HAWKI and ask it to explain the program. Compare its explanation with your own description from Task 3. Where they differ, find out who is right.
b) Ask HAWKI to review `pi_threads.c` for correctness (e.g., race conditions, shared variables) and performance. Check every claim yourself, e.g., by compiling and running a suggested change.
c) Give HAWKI your measurements from Task 3 and ask why the speedup is not ideal. Test at least one of its explanations with an additional measurement.